

Nguyễn Xuân An – ITITI2201

Link github: <https://github.com/anhang204/Lab05/tree/main/StudentManagement>

Question 1: List: Navigate to /student - should see existing students



Student Management System

MVC Pattern with Jakarta EE & JSTL

[+ Add New Student](#)

ID	STUDENT CODE	FULL NAME	EMAIL	MAJOR	ACTIONS
5	SV005	David Wilson	david.w@email.com	Computer Science	Edit Delete
4	SV004	Sarah Davis	sarah.d@email.com	Data Science	Edit Delete
3	SV003	Michael Brown	michael.b@email.com	Software Engineering	Edit Delete
2	SV002	Emily Johnson	emily.j@email.com	Information Technology	Edit Delete
1	SV001	John Smith	john.smith@email.com	Computer Science	Edit Delete

When the user first visits the /student URL.

- Controller (StudentController.java):** The `@WebServlet("/student")` annotation catches this request.
- doGet():** The `doGet()` method is called. Since `request.getParameter("action")` is null, the action variable defaults to "list".
- switch (action):** The switch statement calls the `listStudents(request, response)` method.
- listStudents():**
 - This method calls `studentDAO.getAllStudents()` to get the list of students.
 - StudentDAO.java:** The `getAllStudents()` method connects to the DB and executes the SQL query `SELECT * FROM students ORDER BY id DESC`. It loops through the `ResultSet` and returns a `List<Student>`.
 - listStudents() (continued):** The list of students is attached to the request: `request.setAttribute("students", students)`.
 - Finally, it forwards the request to the view: `RequestDispatcher dispatcher = request.getRequestDispatcher("/views/student-list.jsp")`.
- View (student-list.jsp):**
 - The JSP page receives the students attribute.
 - The `<c:choose>` tag checks: `<c:when test="{not empty students}">`.
 - Since the list is not empty, it iterates (`<c:forEach var="student" items="{students}">`) and displays each student in a table row (`<table>`).

Question 2: Add: Click "Add New Student"

- Fill form with test data
- Submit
- Should redirect to list with success message

+ Add New Student

Student Code *

Format: 2 letters + 3+ digits

Full Name *

Email *

Major *

+ Add Student

✕ Cancel

Student Management System

MVC Pattern with Jakarta EE & JSTL

✓ Student added successfully

+ Add New Student

ID	STUDENT CODE	FULL NAME	EMAIL	MAJOR	ACTIONS
7	SV201	Nguyễn Xuân An	annguyenxuan2004@gmail.com	Information Technology	EditDelete
5	SV005	David Wilson	david.w@email.com	Computer Science	EditDelete

Step A: Click "Add New Student"

1. **View (student-list.jsp):** The user clicks the link: `<a href="student?action=new" ...>`.
2. **Controller (StudentController.java):** `doGet()` is called. This time, action is "new".
3. **switch (action):** The switch statement calls `showNewForm(request, response)`.
4. **showNewForm():** This method simply forwards the request to `/views/student-form.jsp`.
5. **View (student-form.jsp):**
 - The form page loads.
 - The test `<c:when test="${student != null}">` evaluates to false (because no student attribute was set).
 - Therefore, the `<c:otherwise>` block is executed, showing the title "✚ Add New Student".
 - The hidden action input's value is set to insert: `<input type="hidden" name="action" value="${student != null ? 'update' : 'insert'}">`.

Step B: Fill Form and Submit

1. **View (student-form.jsp):** The user fills in the data and hits Submit. The form submits via `method="POST"` to the student action.
2. **Controller (StudentController.java):** This time, `doPost()` is called.
3. **doPost():** It reads the action parameter (which is "insert" from the hidden input).
4. **switch (action):** The switch statement calls `insertStudent(request, response)`.
5. **insertStudent():**
 - It gets all the form data (studentCode, fullName, email, major) using `request.getParameter()`.
 - It creates a new Student object: `Student newStudent = new Student(...)`.
 - It calls `studentDAO.addStudent(newStudent)`.
 - **StudentDAO.java:** The `addStudent()` method executes the SQL: `INSERT INTO students (...) VALUES (?, ?, ?, ?)`.
 - **insertStudent() (continued):** If `addStudent()` returns true, it sends a redirect: `response.sendRedirect("student?action=list&message=Student added successfully")`.

Step C: Redirect with Success Message

1. **Controller & View:** The browser receives the redirect command and re-requests the `/student` URL, which executes **Step 1 (List)** again.
2. **student-list.jsp:** When the list page reloads, it checks the URL parameters.
3. The tag `<c:if test="${not empty param.message}">` is now true.
4. It displays the success message:  `${param.message}`.

Question 4: Delete: Click "Delete" on test student

- Should confirm

- Should redirect with delete message

✎ Edit Student

Student Code *

Format: 2 letters + 3+ digits

Full Name *

Email *

Major *

✎ Edit Student

Student Code *

Format: 2 letters + 3+ digits

Full Name *

Email *

Major *

📖 Student Management System

MVC Pattern with Jakarta EE & JSTL

✔ Student updated successfully

[+ Add New Student](#)

ID	STUDENT CODE	FULL NAME	EMAIL	MAJOR	ACTIONS
7	SV201	Nguyễn Xuân An	ititiu22001@student.hcmiu.edu.vn	Information Technology	✎ Edit 🗑 Delete

Step A: Click "Edit"

1. **View (student-list.jsp):** The user clicks the link: `<a href="student?action=edit&id=${student.id}" ...>`.
2. **Controller (StudentController.java):** `doGet()` is called. The action is "edit", and an id is also present.
3. **switch (action):** The switch statement calls `showEditForm(request, response)`.
4. **showEditForm():**
 - It gets the id from `request.getParameter("id")`.
 - It calls `Student existingStudent = studentDAO.getStudentById(id)`.
 - **StudentDAO.java:** `getStudentById()` runs the SQL: `SELECT * FROM students WHERE id = ?`.
 - **showEditForm() (continued):** It sets the found student as a request attribute: `request.setAttribute("student", existingStudent)`.
 - It forwards to `/views/student-form.jsp`.

Step B: Form is Pre-filled

1. **View (student-form.jsp):**

- The form page loads.
- `<c:when test="${student != null}">` is now true.
- The title is set to "✎ Edit Student".
- The input fields are pre-filled with the student's data using, for example, `value="${student.fullName}"`.
- The hidden action input is set to update, and a hidden input for the id is added: `<input type="hidden" name="id" value="${student.id}">`.

Step C: Modify Data and Submit

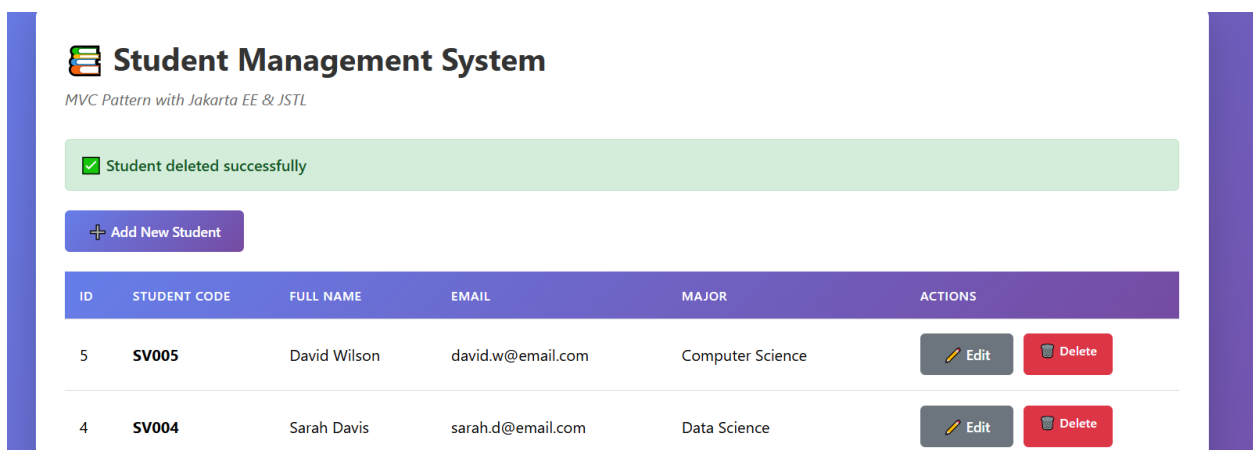
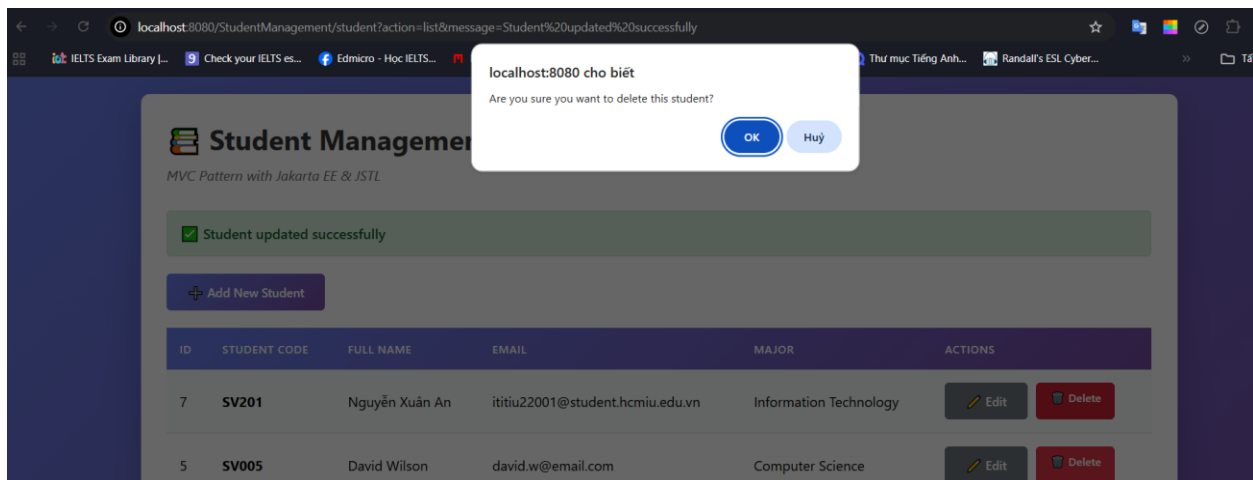
1. **View (student-form.jsp):** The user modifies the data and clicks "Update Student." The form submits via POST to /student.
2. **Controller (StudentController.java):** `doPost()` is called.
3. **`doPost()`:** It reads the action parameter, which is "update".
4. **`switch (action)`:** The switch statement calls `updateStudent(request, response)`.
5. **`updateStudent()`:**
 - It gets *all* the data, including the hidden id.
 - It creates a Student object and sets its ID: `student.setId(id)`.
 - It calls `studentDAO.updateStudent(student)`.
 - **`StudentDAO.java`:** `updateStudent()` runs the SQL: `UPDATE students SET ... WHERE id = ?`.
 - **`updateStudent()` (continued):** If successful, it redirects: `response.sendRedirect("student?action=list&message=Student updated successfully")`.

Step D: Redirect with Update Message

1. This process is identical to **Step 2 (Part C)**.
2. The student-list.jsp page reloads and displays the successful update message.

Question 4: Delete: Click "Delete" on test student

- **Should confirm**
- **Should redirect with delete message**



- View (student-list.jsp):**
 - The user clicks the link: `<a href="student?action=delete&id=${student.id}" ...>`.
 - The JavaScript `onclick="return confirm('...')"` displays a confirmation dialog.
- Controller (StudentController.java):** If the user confirms, `doGet()` is called. The action is "delete".
- switch (action):** The switch statement calls `deleteStudent(request, response)`.
- deleteStudent():**
 - It gets the id from `request.getParameter("id")`.
 - It calls `studentDAO.deleteStudent(id)`.
 - StudentDAO.java:** `deleteStudent()` runs the SQL: `DELETE FROM students WHERE id = ?`.
 - deleteStudent() (continued):** If successful, it redirects:
`response.sendRedirect("student?action=list&message=Student deleted successfully")`.
- View (student-list.jsp):** The list page reloads and displays the successful deletion message.

Question 5: Empty State: Delete all students

- Should show "No students found" message

Student Management System

MVC Pattern with Jakarta EE & JSTL

✓ Student deleted successfully

+ Add New Student



No students found

Start by adding a new student

1. **Logic:** After all students have been deleted (via **Step 4**), the user views the /student page.
2. **Controller (StudentController.java): Step 1 (List)** is executed.
3. **listStudents():**
 - It calls studentDAO.getAllStudents().
 - **StudentDAO.java:** getAllStudents() runs SELECT * ... but the ResultSet is empty.
 - It returns an empty List<Student>.
4. **Controller (continued):** listStudents() sets this *empty* list as a request attribute and forwards to student-list.jsp.
5. **View (student-list.jsp):**
 - The <c:choose> tag is evaluated.
 - <c:when test="{not empty students}"> is now false.
 - Therefore, the <c:otherwise> block is executed.
 - This displays the "empty-state" div, which contains the message "No students found".